Who Runs the Loop?

From Chat Completions to Responses API

Building agentic AI apps with Microsoft Agent Framework

In C# and Python — what changes, what doesn't, and how to design for it

Udaiappa Ramachandran (Udai)

CTO|CSO @ Akumina Inc. | Microsoft MVP (AI)

Web: https://udai.io

LinkedIn: https://linkedin.com/in/udair

Meetup: https://meetup.com/nashuaug



Agenda

- Why this is happening — where orchestration, tools and state live now
- The two APIs side by side, and a status check for September 2026
- Conversation state, reasoning continuity and streaming
- Hosted tools and remote MCP — who runs the loop, who owns identity
- Will everyone adopt Responses? Microsoft Agent Framework as the hedge
- C# and Python in practice: the lines that actually change
- Multi-agent workflows and a Responses-first architecture
- Migration checklist, live demo, and where this goes next

Why this topic matters

Agentic apps move orchestration, tool execution and state out of your application.

- **Your loop, your problem** — on Chat Completions the tool cycle is your code
- **The execution boundary moved** — hosted tools and MCP run in the runtime
- **State became a service** — the service can hold the conversation for you
- **Reasoning became an artifact** — models can carry thinking between turns
- **Multi-agent needs a coordinator** — one call cannot route or hand off



The pattern repeats: execution moves

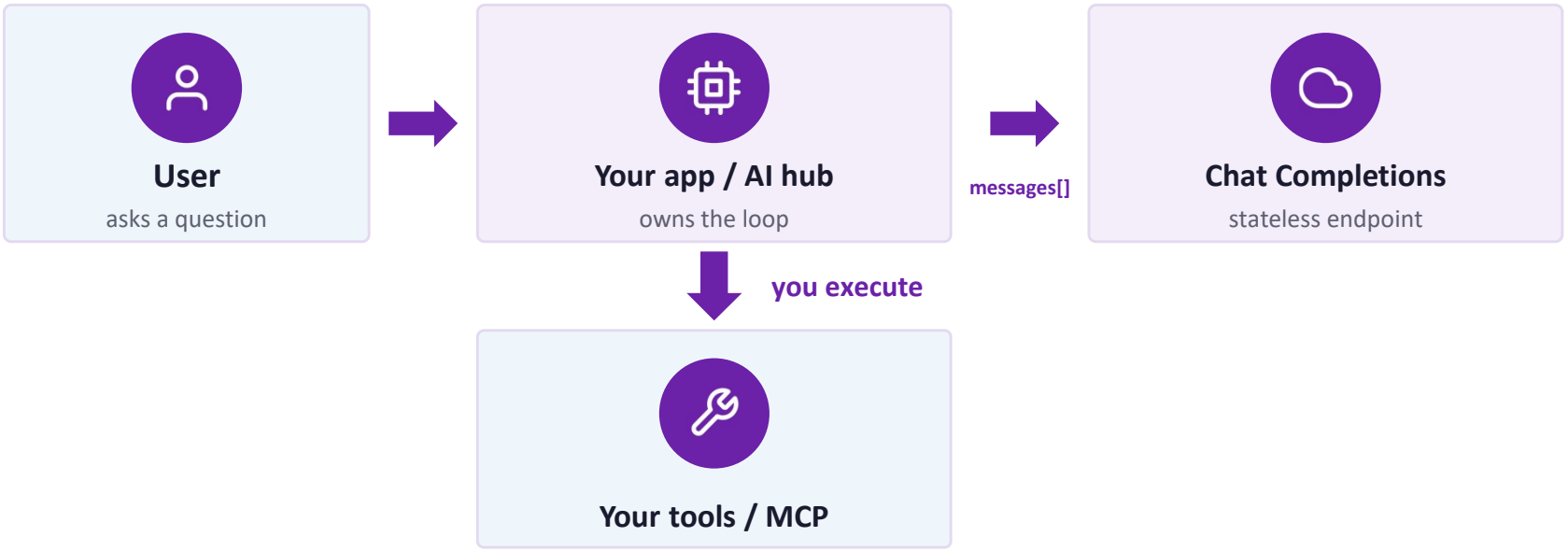
Where code runs has swung back and forth before. It is swinging again.



AI will not remove servers — it redistributes work to wherever cost, latency and policy say it belongs.

The old pattern: Chat Completions

Conversation-first: you send a message array, and your app owns the tool loop.



Great for prompt → answer, classic RAG and simple function calling. The orchestration is yours.

Where Chat Completions gets hard

The tool loop

Receive tool_calls, execute, append the result, call again — every hop is your code.

Remote MCP

Your app must be the MCP client: list tools, invoke them, shuttle results back.

Many agents

You design routing, handoffs, retries and termination conditions yourself.

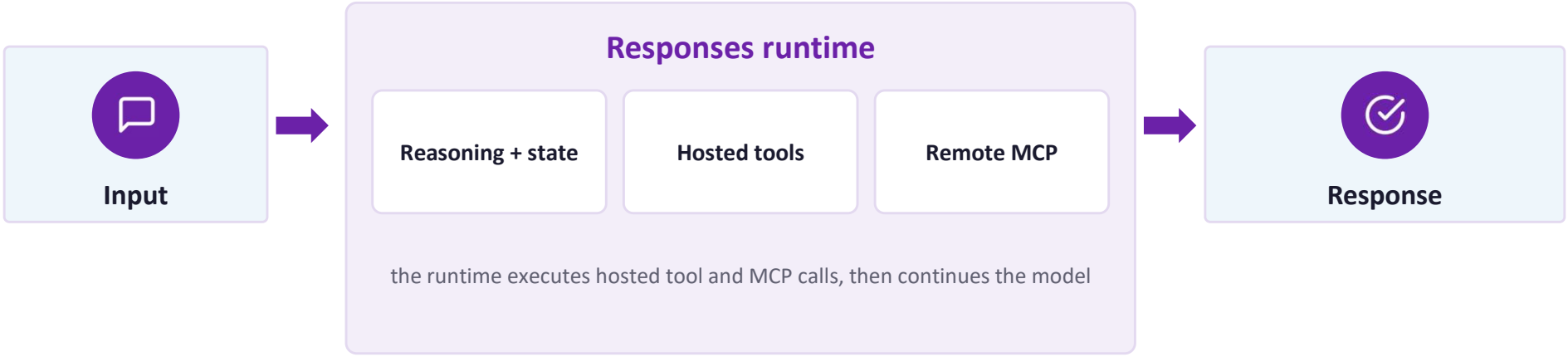
State

You store and replay the whole message array on every single turn.

None of this is impossible on Chat Completions — it is orchestration you write, own and maintain.

The new pattern: Responses API

One request carries input, reasoning, tools and state — and the runtime can close the loop.



Responses enables agentic behaviour — it does not force every app to become an agent.

Chat Completions vs Responses — a fair framing

Chat Completions

- Conversation-first: you send messages[]
- Fully supported — no announced sunset
- Your app owns the tool loop
- Newest models: tools and reasoning cannot be combined
- Great for prompt → answer and classic RAG

Responses API

- Agentic-first: input, tools, state, reasoning
- Recommended by OpenAI for new projects
- Runtime can execute hosted tools and MCP
- Server-side conversations, reasoning reuse
- Typed streaming events instead of raw deltas

Chat Completions can build agents. Responses is designed for agentic applications.
That distinction is the whole talk — not "old vs new".

Status check — where things actually stand (Oct 2026)

Component	Status today	What it means for you
OpenAI Responses API	**Recommended for new projects**	Default choice for new agentic apps
OpenAI Chat Completions	Supported, no sunset date	Existing code keeps working
OpenAI Assistants API	Officially shut down 26 Aug 2026	Use Responses + Conversations
Azure OpenAI Responses	**On the v1 GA endpoint**	/openai/v1/ path, 30+ regions
Azure OpenAI Assistants	Retired	Move to Microsoft Foundry Agents
Microsoft Agent Framework	**.NET & Python GA, Go preview**	Core stable; integrations preview

Retirements happen per model snapshot, not per endpoint — and the dates differ by cloud: OpenAI shuts its legacy gpt-3.5 / gpt-4 / gpt-4-turbo snapshots down on 23 Oct 2026, Azure retired its equivalents earlier. The Chat Completions endpoint itself has no retirement date.

Verify this slide the week you present it — every row here moved at least once in the last year.

Conversation state: three ways, three bills

Chat Completions gives you one option. Responses gives you three — and they cost differently.

Replay it yourself

- You append prior output items
- Works with store: false
- Full context billed every turn

previous_response_id

- Chain one response to the next
- Reasoning stays available across turns
- Whole chain billed as input

Conversations API

- Create once, pass conversation
- Items prepended for you
- Not subject to the 30-day TTL

Responses are stored for 30 days by default (store: true); conversation objects persist until you delete them.
Chat Completions is stateless in context, but new accounts store completions too — storage and threading are different questions.

Reasoning continuity: the part you cannot fake

Keeping reasoning items between turns is what Chat Completions structurally cannot do.

+3%

SWE-bench, same prompt and setup

40–80%

better prompt-cache utilisation

0

extra round-trips to rebuild context

- **Stateless / ZDR** — encrypted reasoning items are returned by default under store: false
- **OpenAI and Azure now match** — neither requires reasoning.encrypted_content in include

Both figures are OpenAI internal evals on SWE-bench and their own cache tests — quote them as vendor claims.

Streaming: raw deltas vs semantic events

Same tokens. Very different client code.

Chat Completions — generic chunks

```
data: {"choices":[{"delta":
        {"content":"Hel"}}]}
data: {"choices":[{"delta":
        {"content":"lo"}}]}
data: [DONE]

// you parse deltas and infer
// what the model is doing
```

Responses — typed events

```
event: response.created
event: response.output_item.added
event: response.output_text.delta
event: response.completed

// subscribe to the events you
// care about — each has a
// predefined schema
```

File-search state and code-interpreter steps arrive as typed events; MCP approvals arrive as an mcp_approval_request item. No string parsing, and no guessing what the runtime is doing on your behalf.

Hosted tools: what the runtime can run for you

Web search

Current web results, with domain and source controls.

File search

Retrieve from vector stores that the provider manages.

Code interpreter

Runs sandboxed Python for analysis and file output.

Remote MCP

The runtime acts as the MCP client, not your app.

Shell

Hosted by OpenAI, or your own runtime. Supersedes local_shell.

apply_patch

Model proposes diffs; your harness applies them.

Declared in the request and executed by the provider — except apply_patch, where the model proposes and you apply.
Hosted does not mean unrestricted: you still declare exactly which tools each agent may use.

Where each tool actually runs

Microsoft Agent Framework's provider support matrix — Python clients.

Tool type	Responses	Chat Completions	Foundry	Anthropic
Function tools	✓	✓	✓	✓
Local MCP tools	✓	✓	✓	✓
Web search	✓	✓	✓	✓
Code interpreter	✓	—	✓	✓
Hosted MCP tools	✓	—	✓	✓
File search	✓	—	✓	—
Image generation	✓	—	✓	—
Hosted shell	✓	—	—	—

C# differs: no web search on Foundry or Anthropic, no code interpreter on Anthropic, and no image generation or hosted shell rows. The Go SDK differs again.

Chat Completions has the smallest hosted-tool surface. Foundry and Anthropic cover much of the same ground. Responses is not unique per tool — it combines the broadest hosted-tool surface under one API.

Remote MCP vs Hosted MCP

"Remote" describes where the server runs. "Hosted" describes who acts as the MCP client.

Local / remote MCP client

- Your app connects to the MCP server
- You list tools and expose them as functions
- You execute each call and return the result
- Works on Chat Completions and Responses

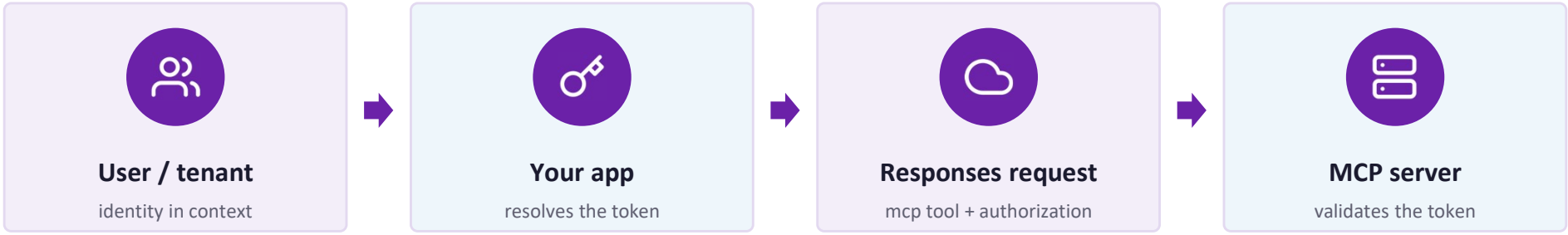
Hosted MCP

- You declare the server in the request
- The provider runtime is the MCP client
- It lists, calls and folds results into context
- Responses, Foundry and Anthropic clients

Same protocol, different client. Hosted MCP takes your app out of the middle of every tool call.

Hosted MCP: identity, approval, allow-lists

The model never picks the credentials. Your app or identity system does.



- **require_approval** — "never" | "always" | a per-tool object; approvals arrive as mcp_approval_request items
- **allowed_tools** — trim the server's tool list to cut latency, cost and blast radius
- **authorization** — not stored by OpenAI, so resend it every request; Foundry Agent Service uses project connections

The model chooses the tool. Your application chooses the authority the tool runs under.

How many times is your app in the loop?

Count the exchanges your application has to take part in.

App-managed tool loop

1. App → model: messages + tool schemas
2. Model → app: tool_calls
3. App → tool or MCP server
4. App → model: tool result
5. Model → app: final answer

4 app-side exchanges

Hosted tool loop

1. App → Responses: input + hosted tools
2. Runtime → tool or MCP server
3. Runtime continues the model
4. Responses → app: final answer

1 app-side exchange

Fewer exchanges is not only latency — it is less retry logic, less state to hold, and less to get wrong.

Measured in the demo: 3 HTTP requests from the app per turn on Chat Completions, 1 on Responses.

Guardrails: expose only the tools needed

Built-in tools do not mean unrestricted tools — exposure is decided per agent.

Knowledge chat

- Allowed: file search only
- No web, no HR, no code
- Deterministic retrieval

Employee assistant

- Allowed: file search + HR MCP
- Approval on any write
- Token scoped to the user

Data analyst

- Allowed: search + code
- Sandboxed container only
- No external side effects

Deterministic RAG: your app retrieves, then asks. Controlled agentic RAG: the model may call an approved retrieval tool. Same API, same guardrails — you pick the level of autonomy per scenario.

Will everyone adopt Responses?

The Responses protocol is spreading faster than its full runtime semantics.

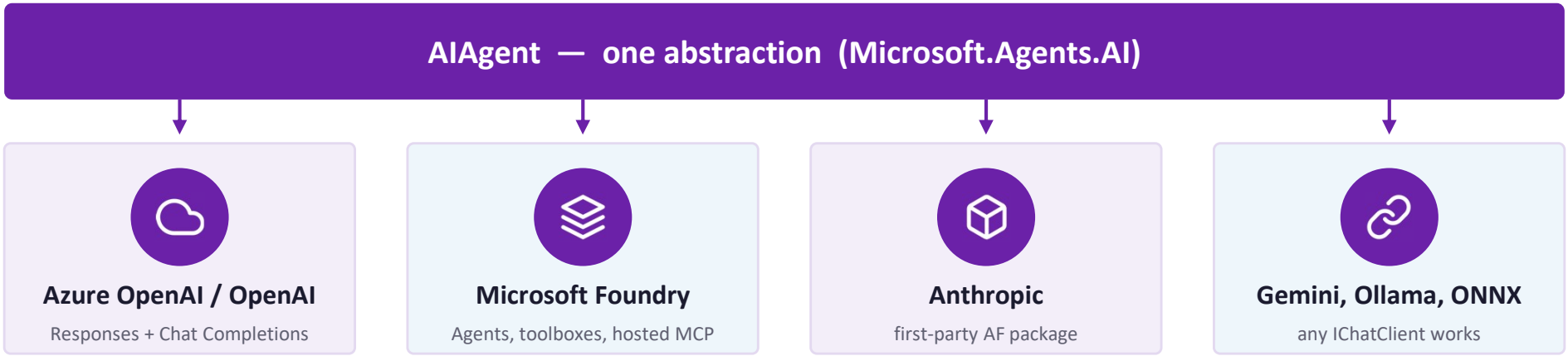
Surface	Speaks /v1/responses	Server-side state
OpenAI / Azure OpenAI	**Native**	**previous_response_id + store**
AWS Bedrock	Yes — selected models	State on both; tools on bedrock-mantle
xAI, Fireworks	Yes — native	store: true by default
Groq	Yes	No — store / previous_response_id
OpenRouter	Yes (beta)	No — stateless by design
vLLM, SGLang, Ollama	Yes	None, or experimental
Anthropic	No — Messages API	Managed Agents (beta)
Google Gemini	No — Interactions API (beta)	previous_interaction_id

- **OpenAI's Conversations API** — little third-party adoption; on Azure it arrives via Foundry projects
- **Every lab shipped its own** — Responses, Anthropic Managed Agents, Google Interactions
- **The real standards are one layer up** — MCP and A2A, both Linux Foundation governed

Same wire shape is not the same runtime semantics — which is why the abstraction earns its place.

Microsoft Agent Framework: one abstraction, many providers

Keep your product code above provider-specific protocols.



- **Sessions, middleware, telemetry** — the same code whichever provider is underneath
- **Workflows GA** — Anthropic GA in .NET, beta in Python; A2A, DevUI and Foundry still preview

.NET 1.25 and Python 1.22 are GA; Go is in public preview. Python sub-packages version separately.

C# — the lines that actually change

Same AIAgent, same tools, same session API. Only the client construction differs.

Chat Completions client

```
using OpenAI;
using OpenAI.Chat;

AIAgent agent = new OpenAIClient(
  new BearerTokenPolicy(cred, scope),
  new OpenAIClientOptions {
      Endpoint = endpoint })
    .GetChatClient(deployment)
    .AsAIAgent(
      instructions: prompt,
      tools: tools);
```

Responses client

```
using OpenAI;
using OpenAI.Responses;

AIAgent agent = new OpenAIClient(
  new BearerTokenPolicy(cred, scope),
  new OpenAIClientOptions {
      Endpoint = endpoint })
    .GetResponsesClient()
    .AsAIAgent(model: deployment,
      instructions: prompt,
      tools: tools);
```

One package — OpenAI 2.14 — pointed at your Azure resource's /openai/v1/ endpoint. No Azure.AI.OpenAI prerelease needed. Only the client line differs: GetChatClient(deployment) → GetResponsesClient() + AsAIAgent(model: …). AgentThread is now AgentSession.

Python — the same change, and the name to watch

One class name changes. That class name is also the trap.

Chat Completions client

```
from agent_framework import Agent
from agent_framework.openai import \
    OpenAIChatCompletionClient

agent = Agent(
  client=OpenAIChatCompletionClient(
    model=deployment,
    azure_endpoint=endpoint,
    credential=AzureCliCredential()),
  instructions=prompt,
  tools=tools)
```

Responses client

```
from agent_framework import Agent
from agent_framework.openai import \
    OpenAIChatClient

agent = Agent(
  client=OpenAIChatClient(
    model=deployment,
    azure_endpoint=endpoint,
    credential=AzureCliCredential()),
  instructions=prompt,
  tools=tools)
```

Watch out: OpenAIChatClient is the Responses client. Chat Completions is OpenAIChatCompletionClient.
The AzureOpenAI classes were removed — pass azure_endpoint and credential to the same client. ChatAgent is now Agent.*

Hosted MCP in a handful of C#

Declare the server; the Responses runtime does the calling.

```
var mcpTool = new HostedMcpServerTool(
    serverName: "microsoft_learn",
    serverAddress: "https://learn.microsoft.com/api/mcp")
{
    AllowedTools = ["microsoft_docs_search"],
    ApprovalMode = HostedMcpServerToolApprovalMode.NeverRequire
};

AIAgent agent = new OpenAIClient(tokenPolicy, options)
    .GetResponsesClient()
    .AsAIAgent(model: deployment, tools: [mcpTool]);
```

- **AllowedTools** — narrow the server's tool list to what this agent is permitted to use
- **ApprovalMode** — NeverRequire, AlwaysRequire, or RequireSpecific(always, never) per tool
- **Foundry toolboxes** — HostedMcpToolboxAITool exposes a managed tool set over one MCP endpoint

Hosted MCP in Python

No class for hosted MCP — the Responses client hands you the tool.

```
from agent_framework import Agent
from agent_framework.openai import OpenAIChatClient
client = OpenAIChatClient(model=deployment, azure_endpoint=endpoint,
                          credential=AzureCliCredential())
mcp_tool = client.get_mcp_tool(
    name="Microsoft Learn MCP",
    url="https://learn.microsoft.com/api/mcp",
    allowed_tools=["microsoft_docs_search"],
    approval_mode="never_require")
agent = Agent(client=client, instructions=prompt, tools=mcp_tool)
```

- **get_mcp_tool** — a static factory on the Responses client; it returns the tool dict for you
- **approval_mode** — "never_require" | "always_require", or {"never_require_approval": [tools]}
- **App-side MCP has classes** — MCPStreamableHTTPTool and MCPStdioTool, when you run the loop

Sessions and state in C#

Service-managed by default; local when you need it. store:false needs NoWarn MAAI001.

```
// Service-managed state (the default on a Responses client)
AgentSession session = await agent.CreateSessionAsync();
await agent.RunAsync("Summarise the travel policy.", session);
await agent.RunAsync("Now list the exceptions.", session);

// Same agent, history kept locally: store = false
AIAgent local = client.GetResponsesClient()
    .AsIChatClientWithStoredOutputDisabled(model: deployment)
    .AsAIAgent(instructions: prompt);
```

- **SerializeSessionAsync / DeserializeSessionAsync** — persist and rehydrate a session
- **Chat Completions clients** — in-memory or custom history only; no service-managed option exists
- **Foundry** — CreateSessionAsync(conversationId) binds to an existing server conversation

Reasoning still chains under store: false — the encrypted reasoning items travel with the request.

Sessions and state in Python

Service-managed by default on the Responses client; one keyword flips it local.

```
from agent_framework import Agent, AgentSession

agent = Agent(client=OpenAIChatClient(...), instructions=prompt)

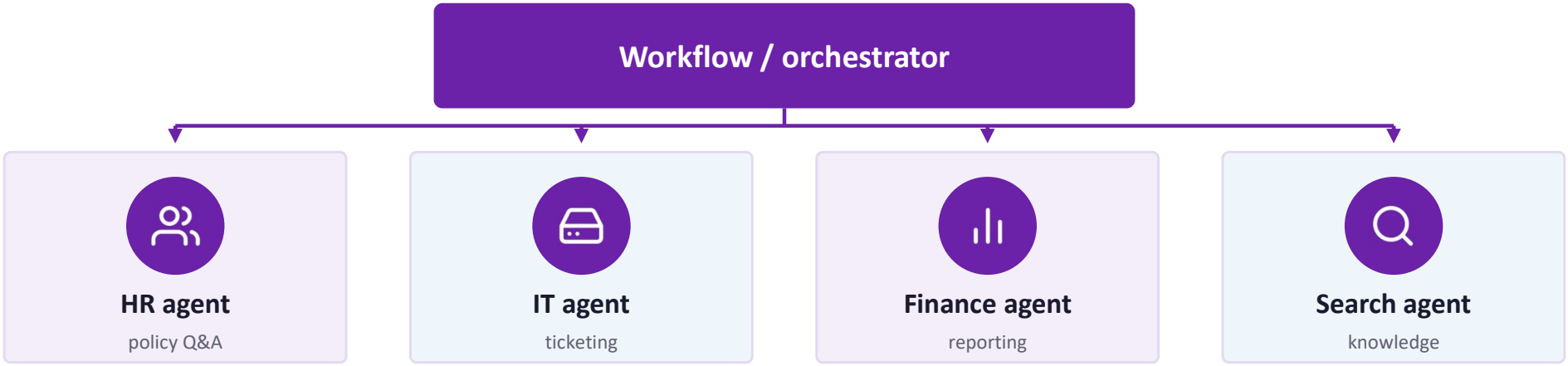
session = agent.create_session()          # sync — no await
await agent.run("Summarise the travel policy.", session=session)
await agent.run("Now list the exceptions.", session=session)

blob = session.to_dict()                  # persist anywhere
resumed = AgentSession.from_dict(blob)
# keep history locally instead of in the service
await agent.run("...", session=resumed, options={"store": False})
```

- **create_session() is synchronous** — Python does not await it, unlike .NET's CreateSessionAsync
- **to_dict / from_dict** — the serialisation pair; there is no serialize_session helper
- **options={"store": False}** — the agent auto-injects an in-memory history provider instead

Multi-agent: Responses is the runtime, workflows coordinate

Responses runs the model and its tools. Coordination is a different layer.



AgentWorkflowBuilder: sequential · concurrent · group chat · handoff · magentic. WorkflowBuilder adds checkpoints, sub-workflows, conditional edges and human-in-the-loop.

Microsoft.Agents.AI.Workflows is GA — orchestration is no longer the experimental part.

Recommended product architecture

Responses-first, with Chat Completions as the compatibility adapter.

Your AI service abstraction (AIAgent)

Responses — primary path

- Hosted tools and remote MCP
- Service-managed conversations
- Reasoning reuse and typed events

Chat Completions — compatibility

- OpenAI-compatible providers and BYOM
- Local and self-hosted models
- Function tools and local MCP

Do not expose "Responses vs Chat Completions" as a customer setting.
Treat it as a provider capability your abstraction resolves at runtime.

Migration checklist

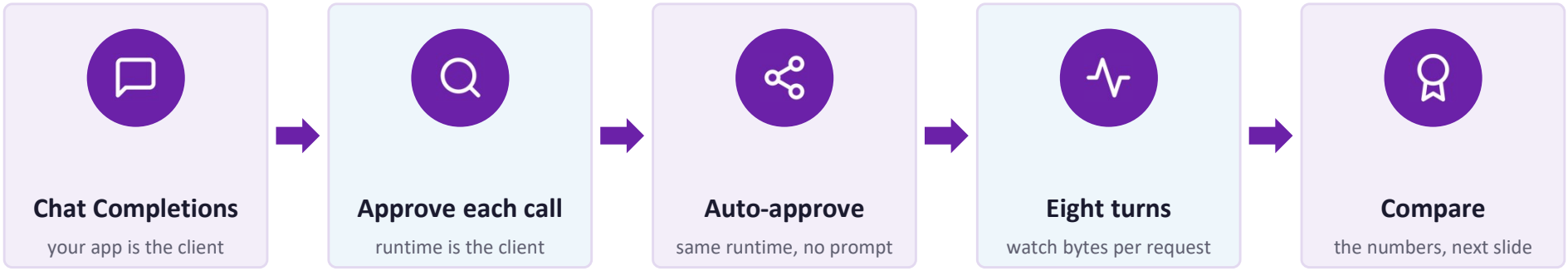
What to actually do on Monday morning.

- **Swap the client, not the app** — GetResponsesClient() + AsAIAgent(model: …)
- **Move the loop to the runtime** — where hosted execution is available
- **Pick a state strategy** — session, previous_response_id, or Conversations
- **Declare tools per agent** — allow-lists, approval on side effects
- **Move MCP clients server-side** — HostedMcpServerTool where supported
- **Re-wire streaming** — typed events instead of raw deltas
- **Check the provider matrix** — Azure and Foundry differ from OpenAI direct
- **Add telemetry** — AsBuilder().UseOpenTelemetry(…) before you scale



Demo storyboard

One KB question, one MCP tool, three ways to run the loop.



The demo point is not the answer. It is how little of the loop is left in the application.

What the demo measured

Same model, same MCP server, same 8 questions. Only the API changes.

Over 8 turns	Chat Completions	Responses + hosted MCP
Model requests from the app	16	8
MCP calls from the app	8	**0**
Bytes sent to the model	129,695	**6,453**
Request size, turn 1 → turn 8	3.1 KB → 28.0 KB	**0.7 KB → 0.8 KB**
Total time	26.0 s	27.8 s

gpt-5.6-terra on Azure OpenAI, MCP server on Azure Container Apps, single run, approval off. Fewer bytes is not fewer tokens — previous_response_id still bills the whole chain as input.

20× less traffic and no tool code in the app, at the same speed.

Is Responses faster? Not by itself

The work does not disappear. It moves inside the provider.

Chat Completions

- App → model: model picks the tool
- App → MCP: connection already open
- App → model: model writes the answer

Responses + hosted MCP

- Runtime lists MCP tools (~150 ms, every turn)
- Model picks the tool
- Runtime → MCP: tool runs
- Model writes the answer

Where Responses wins on time

- Slow or distant clients — mobile, browser, cross-region, where a saved trip costs hundreds of ms
- Many tool calls per turn — each one is a round trip through your app on Chat Completions
- Large tool results — Chat Completions re-uploads them on every later request
- Reasoning models — reasoning carries across turns instead of being rebuilt

Measure speed on your own network. Choose Responses for the loop you no longer have to run.

Where this goes: hybrid execution

Cost, privacy, latency and policy keep pushing execution around.



Device / edge

- Small, cheap reasoning
- Sensitive local data
- Offline paths



Your backend

- Business rules
- Identity and policy
- Audit and approval



Provider runtime

- Frontier reasoning
- Hosted tools
- Remote MCP



External systems

- SaaS APIs
- Enterprise data
- Partner agents

Servers do not disappear. They keep the authority while execution spreads out.

Resources & Q&A

OpenAI — migrate to Responses

developers.openai.com/api/docs/guides/migrate-to-responses

OpenAI — conversation state

developers.openai.com/api/docs/guides/conversation-state

OpenAI — tools, connectors & MCP

developers.openai.com/api/docs/guides/tools-connectors-mcp

OpenAI — streaming responses

developers.openai.com/api/docs/guides/streaming-responses

Azure OpenAI — Responses API

learn.microsoft.com/azure/foundry/openai/how-to/responses

Agent Framework — tools matrix

learn.microsoft.com/agent-framework/agents/tools/

Agent Framework — hosted MCP

learn.microsoft.com/agent-framework/agents/tools/hosted-mcp-tools

Model providers (C# / Python)

learn.microsoft.com/agent-framework/integrations/by-component/model-providers/

Agent Framework repo & samples

github.com/microsoft/agent-framework

My samples

github.com/nhcloud/responses-api

THE TAKEAWAY

Chat Completions can build an agent. Responses is built for agentic applications — start there, and keep the abstraction above it.

Thank you! Questions, and let's compare notes on what you are migrating.

Udai — udai.io | linkedin.com/in/udair | meetup.com/nashuaug